Introduction to Parallel and Distributed Computing

Institute of Computer Science & Information Technology,
Faculty of Management & Computer Sciences,
The University of Agriculture, Peshawar, Pakistan.

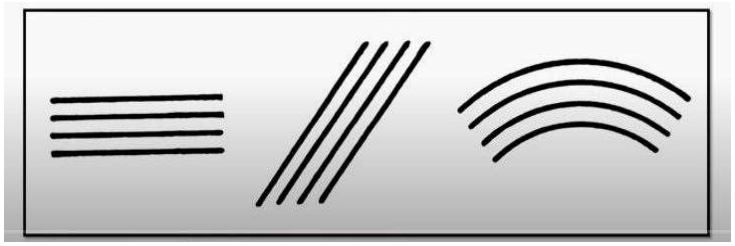
Computing

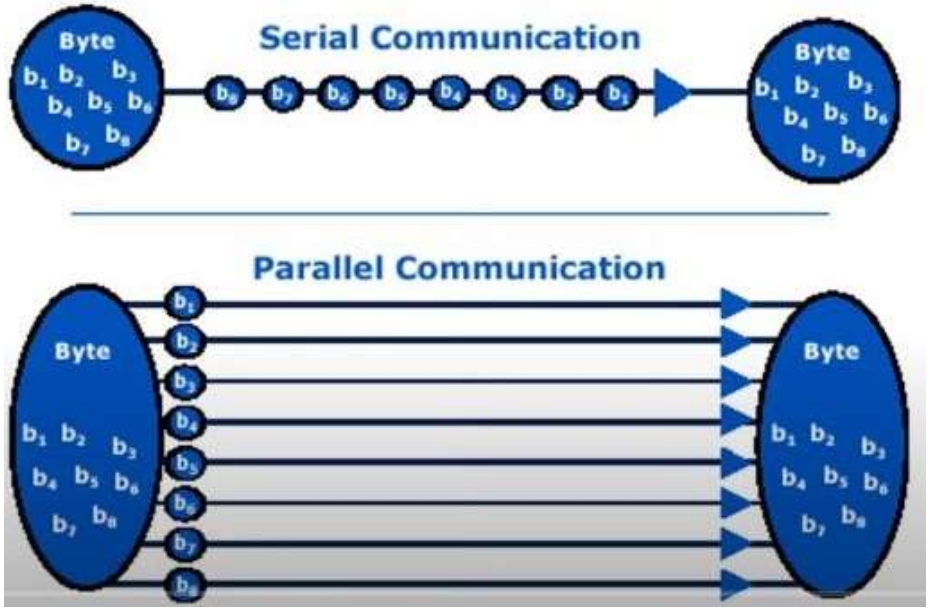
- Computing is the process to complete a given goal-oriented task by using computer technology

- Computing may include the design and development of software and hardware systems for a broad range of applications, often consists of structuring, processing and managing any kind of information

Serial vs Parallel

- Parallel (in mathematics) means two lines that never intersect.





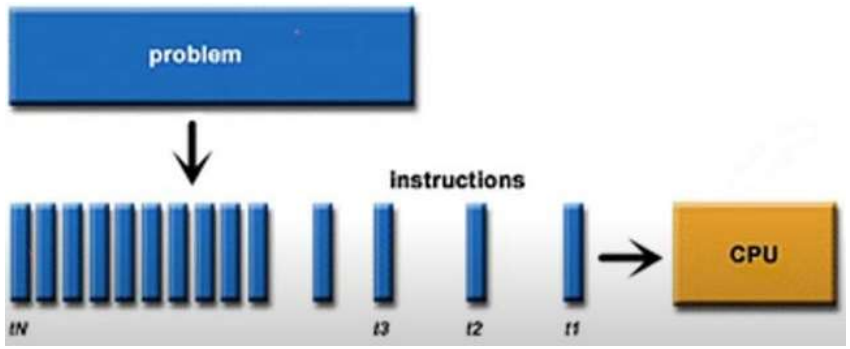
Parallel Computing Systems

- Parallel Computing Systems are the simultaneous execution of the single task (split up and adapted) on multiple processors in order to obtain results faster.

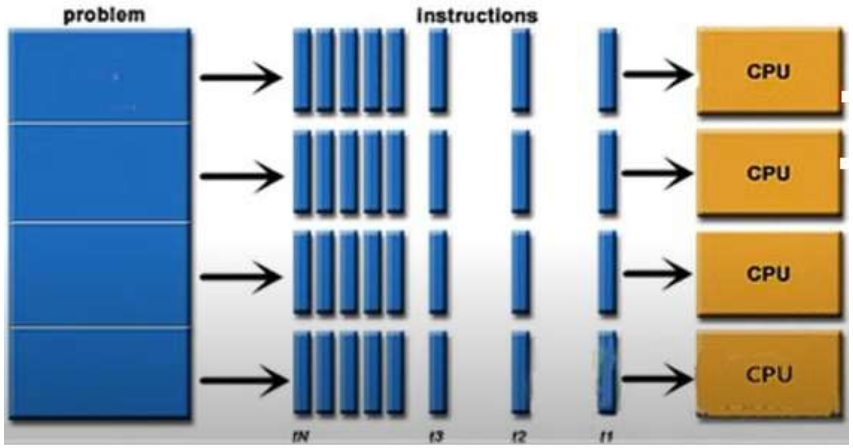
- The idea is based on the fact that the process of solving a problem usually can be divided into smaller tasks (divide and conquer), which may be carried out simultaneously with some coordination.

Cont.

- Serial Computing



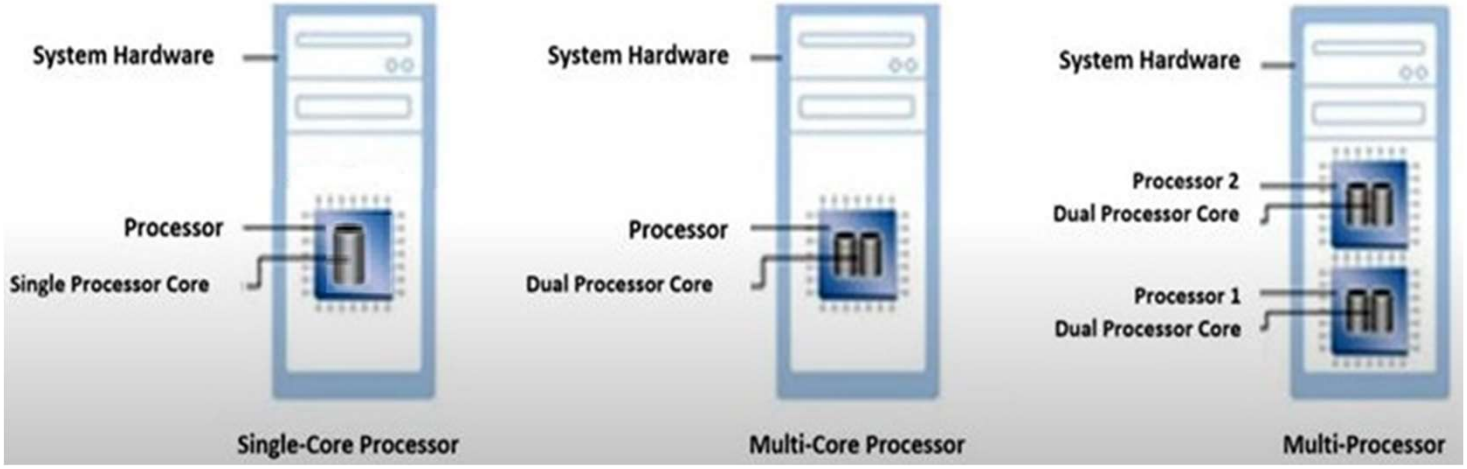
- Parallel Computing



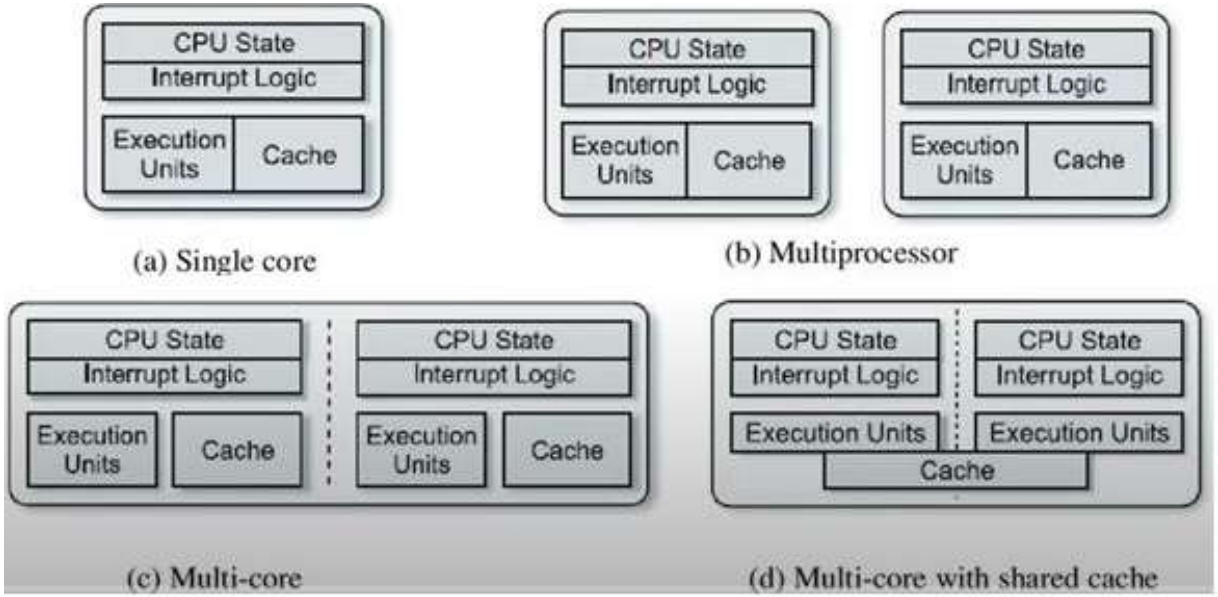
Cont.

- The terms parallel computing architecture sometime used for a computer with more than one processor (few to thousands), available for processing.

- The multi-core processors (chips with more than one processor core) are some commercial examples which bring parallel computing to the desktop.



Multi-Core & Multi-Processor



Why use Parallel Computing?

- In the natural world, many complex, interrelated events are happening at the same time, yet within a temporal sequence.
  - Weather systems (such as rainfall, temperature changes, wind patterns, and cloud formations occur concurrently and are interconnected)
  - Economic systems (numerous transactions, investments, productions, and consumption activities occur simultaneously)
- Compared to serial computing, parallel computing is much better suited for modeling, simulating and complex real world phenomena.

Cont.

Save Time & Money

- In theory, throwing more resources at a task will shorten its time to completion, with potential cost savings (Increase Throughput)
- Parallel computers can be built from cheap, commodity components.

Solve Large / More Complex Problems

- Many problems are so large and complex that it is impractical or impossible to solve them using a serial program, especially given limited computer memory.
- E.g., Web search engines/ databases processing millions of transaction every second.

Concurrency

- A single compute resource can only do one thing at a time.
- Multiple compute resources can do many things simultaneously.
- E.g., Collaborative networks provide a global venue where people from around the world can meet and conduct work "virtually".

Take advantage of non-local resources

- Using compute resources on wide area network, or even the internet when local compute resources are scarce or insufficient.

Better use of underlying Parallel Hardware

- Modern computers, even laptops are parallel in architecture with multiple processors/cores.
- Parallel software is specifically intended for parallel hardware with multiple cores, threads, etc.
- In most cases, serial programs run on modern computers "waste" potential computing power.

Scope of Parallel Computing

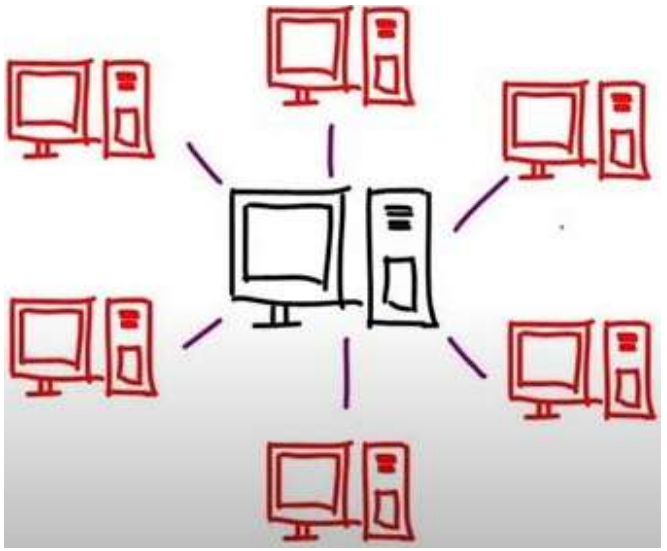
- Parallelism finds applications in very diverse application domains for different motivating reasons.
  - These range from improved application performance to cost considerations
- **Applications in Engineering and Design:**
  - Design of Internal Combustion Engine, Micro and Nano Scale Systems.
- **Scientific Applications:**
  - characterization of genes and proteins
  - Applications in astrophysics have explored the evolution of galaxies
  - Weather modeling, mineral prospecting, flood prediction, etc.
- **Commercial Applications**
  - Data mining and analysis for optimizing business and marketing decisions.
  - Large scale servers (mail and web servers) are often implemented using parallel platforms.

Limitations

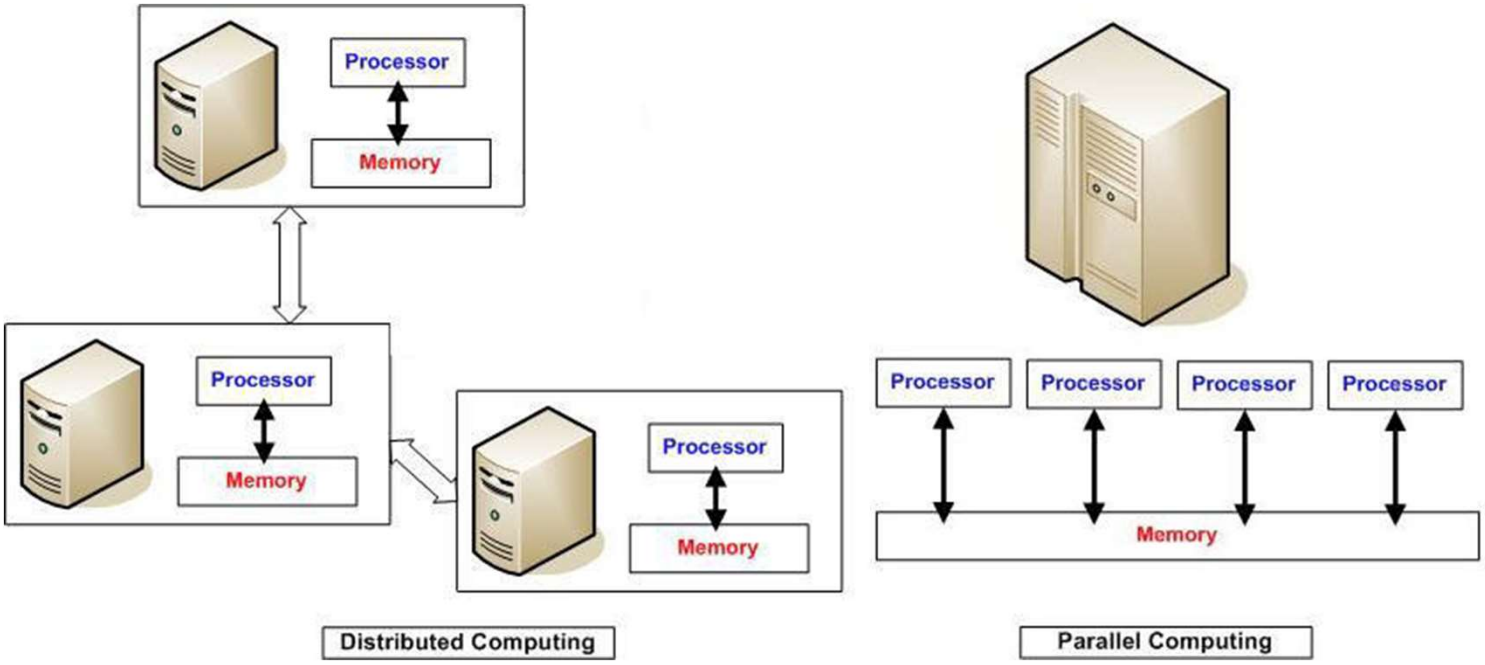
- Memory Access Time
  - Shared memory may cause bottlenecks when multiple processors attempt to access data simultaneously.
  - Inconsistent latency between local and remote memory in distributed systems (NUMA effects).
- Communication Cost
  - Processors spend time exchanging data rather than computing.
  - High latency and bandwidth limitations can reduce efficiency, especially in large-scale systems.
- Data and Control Dependency
  - Tasks may depend on results from others, limiting parallel execution.
  - Control dependencies (e.g., conditional branches) reduce opportunities for parallelization.

Distributed Systems

- Distributed system is defined as one in which hardware or software components located at networked computers communicate and coordinate their actions only by passing messages.
- This simple definition covers the entire range of systems in which networked computers can usefully be deployed.



Cont. (Shared vs Distributed Memory)



Why use Distributed Computing?

- **Historical Reason**: Computing resources that used to operate independently now need to work together.
  - E.g., consider an office that acquired personal workstations for individual use. After a while, there were many workstations in the office building, and the users recognized that it would be desirable to share data and resources among the individual computers.
- **Functional Reason**: if there is special function hardware or software available over the network, then that functionality does not have to be duplicated on every computer system (or node) that needs to access the special-purpose resource.
- **Economical Reason**: It may be more cost-effective to have many small computer working together than one large computer of equivalent power.

Cont.

- **Scalable**: Having many units connected to a network is the more flexible configuration; if more resources are needed, another unit can be added in place, rather than bringing the whole system down and replacing it with an upgraded one.

- **Reliable:** A distributed system can be more reliable and available than a centralized system.
  - This is a result of the ability to replicate both data and functionality.
  - E.g., when a given file is copied on two different machines, than even if one machine is unavailable, the file can still be accessed on the other machine.

Few problem in distributed computing

- E.g., are keeping multiple copies of data consistent, and keeping the clocks on different machines in the system synchronized.
- A system that provides distributed computing support must address these new issues.

Differences (Parallel vs Distributed)

- **Architecture:**
  - Parallel computing typically involves **a single system with multiple processors** or cores working together on a shared memory space to solve a problem.
  - Distributed computing involves **multiple independent systems** (nodes) connected via a network, each with its own memory and processor, working together to solve a problem.
- **Communication:**
  - In parallel computing, communication between processors occurs via **shared memory**, allowing for faster communication and synchronization.
  - In distributed computing, communication between nodes occurs **over a network**, which introduces latency and requires more complex communication protocols.
- **Scalability:**
  - Parallel computing can be **limited by the number of processors** or cores available on a single system, making it **less scalable** compared to distributed computing.
  - Distributed computing can scale more easily by adding more nodes to the network, allowing for **potentially unlimited scalability**.

Cont.

- **Fault Tolerance:**
  - Parallel computing systems **may lack built-in fault tolerance** features, as they **rely on a single system**, making them more **vulnerable to failures**.
  - Distributed computing systems are typically designed with **built-in fault tolerance** mechanisms since nodes can fail independently.
- **Programming Model:**
  - Parallel computing often utilizes **shared-memory programming models** like OpenMP or MPI (Message Passing Interface).
  - Distributed computing commonly employs **message-passing models** like Hadoop, Spark, or MPI, where communication between nodes is explicit.
- **Data Storage:**
  - In parallel computing, **data storage is centralized** and typically shared across all processors or cores.
  - In distributed computing, **data storage is distributed** across multiple nodes, often employing distributed file systems like HDFS (Hadoop Distributed File System) or cloud storage solutions.

Cont.

Sr.	Parallel Computing	Distributed Computing
1	Many operations are performed simultaneously	System components are located at different locations
2	Single computer is required	Uses multiple computers
3	Multiple processors perform multiple operations	Multiple computers perform multiple operations
4	It may have shared or distributed memory	It have only distributed memory
5	Processors communicate with each other through bus	Computer communicate with each other through message passing
6	Improves the system performance	Improve system scalability, fault tolerance and resource sharing capabilities

Scalability

- It is the property of a system to handle a growing amount of work by adding resources to the system.
- A system is described as scalable if it will remain effective when there is a significant increase in the number of resources and the number of users.
- Scalability of a system can be measured along the following different dimensions:

1. Physical/Load Scalability: A system can be scalable with respect to its size, meaning that we can easily add/remove more users and resources to the system.
2. Functional: The ability to enhance the system by adding new functionality without disrupting existing activities.
3. Geographic: The ability to maintain effectiveness during expansion from a local area to a larger area.
4. Generation: The ability of a system to scale by adopting new generations of components.
5. Heterogeneous: is the ability to adopt components from different vendors

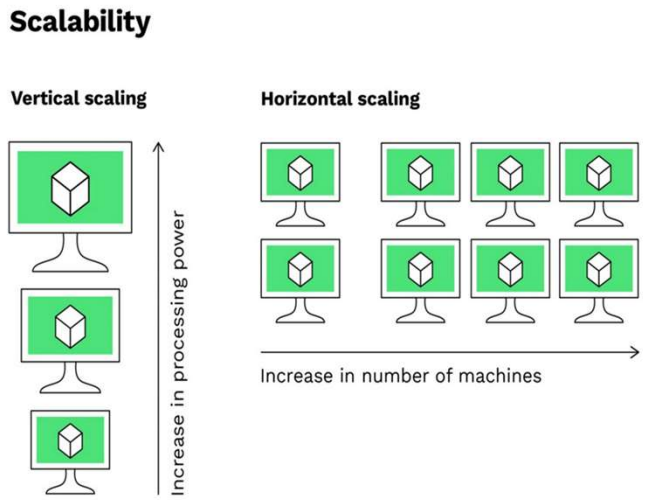
Horizontal vs Vertical Scaling

- Two dimensions of Scaling are Horizontal and Vertical Scaling

- **Scaling Horizontally: (in/out)** Means adding more nodes to (or removing nodes from) a system, such as adding a new computer to a distribute software applications.
  - E.g. scaling out (to increase) from one web server to three.
  - Exploiting this scalability requires software for efficient resource management and maintenance.

- **Scaling Vertically: (up/down)** mean adding more resources to (or removing resources from) a single node, typically involving the addition of CPUs, memory or storage to a single computer.

- Larger number of elements increases management complexity, more sophisticated programming to allocate tasks among resources and handle issues such as throughput and latency across nodes.



Speedup and Amdahl's Law

- Amdahl's law is a formula which gives the theoretical speedup in latency of the execution of a task at fixed workload that can be expected of a system whose resources are improved (scalability).

- It is named after computer scientist Gene Amdahl, and was presented at the AFIPS.

- Amdahl's law is often used in parallel computing to **predict the theoretical speedup** when using multiple processors.

What is Amdahl's Law?

$$Speedup = \frac{time\ to\ execute\ program\ on\ a\ single\ processor}{time\ to\ execute\ program\ on\ N\ parallel\ processors}$$

$$Speedup = \frac{T(1-p)+Tp}{T(1-p)+\frac{Tp}{n}}$$

$$Speedup = \frac{1}{(1-p)+\frac{p}{n}}$$

p – fraction of code that can be run in parallel

n – system resource improvement (number of processors)

Ideal: $\lim_{p \to 1} speedup = n$

Code example OpenMP

```
#include <stdio.h>
#include <omp.h>
#define ARRAY_SIZE 1000
int main() {
    int array[ARRAY_SIZE];
    int sum = 0;
    // Initialize array
    for (int i = 0; i < ARRAY_SIZE; i++) {
        array[i] = i + 1;
    }
    // Parallelize the loop to calculate sum
    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < ARRAY_SIZE; i++) {
        sum += array[i];
    }
    printf("Sum: %d\n", sum);
    return 0;
}
```

- The loop is parallelized using OpenMP pragma ***#pragma omp parallel for,*** which distributes the loop iterations across multiple threads.

- The ***reduction(+:sum)*** clause is used to ensure that each thread has its own private copy of ***sum***, and at the end of the parallel region, all private copies are combined into a single global sum.